

RAG Explained

Retrieval-Augmented Generation — a plain-language guide

1. What is RAG?

A standard LLM is trained on general data up to a cutoff date. It cannot know your company's internal policies, client records, or product specs — it can only guess. RAG fixes this by giving the model a way to look things up before answering. Instead of relying on memorised weights, it retrieves the exact relevant text from your own documents and feeds it as context. The answer is therefore grounded in real, specific, up-to-date information rather than a hallucinated approximation.

2. Ingestion Pipeline (one-time setup)

Documents PDFs, Docs, URLs, etc.	Extract Strip formatting, pull raw text	Text Chunk Split into small pieces	Embed Convert chunks to vectors	Vector DB Store with metadata
--	---	---	---------------------------------------	-------------------------------------

3. Query Pipeline (every question)

User Question	Embed Question	Similarity Search	Retrieved Chunks	LLM + Context	Grounded Answer
------------------	-------------------	----------------------	---------------------	------------------	--------------------

What is our return policy? Turn question into a vector Find nearest vectors in DB Top-k matching text chunks Prompt with real context
Factual, specific response

4. Chunking — and Why Size Matters

Chunking is the step where a long document is split into smaller pieces before embedding. Each chunk becomes one searchable unit in the vector database. Getting the size right is critical:

- **Too small (< 100 tokens):** individual chunks lose context. A sentence about a refund policy means little without the surrounding paragraph that explains when it applies.
- **Too large (> 1000 tokens):** the embedding blurs — many topics get averaged into one vector, making retrieval imprecise. You also risk exceeding the LLM's context window.
- **Sweet spot (~200-500 tokens):** enough context for the embedding to be meaningful, small enough that the retrieved chunk stays focused and relevant.

5. Three Problems RAG Would Solve at Petalnex

1 Client Onboarding Knowledge Gap

New automation workflows at Petalnex often require onboarding clients with bespoke documentation — SOPs, integration guides, FAQ sheets. Without RAG, a support agent must manually search folders. With RAG, the agent types a natural-language question and receives the exact paragraph from the right document instantly, reducing onboarding time and human error.

2 Stale AI Responses on Internal Policies

The n8n workflows and automation rules at Petalnex change frequently. A bare LLM assistant would give answers based on its training, not on the actual current playbook. RAG grounds every answer in the latest uploaded policy files, so responses stay accurate without retraining.

3 Cross-Project Knowledge Retrieval

A data science team running multiple client automation projects accumulates scattered reports, meeting notes, and output summaries. RAG lets any team member query across all projects — e.g. 'what chunking strategy did we use for the retail client?' — and get a sourced, specific answer rather than hunting through shared drives.